

哈尔滨工业大学

编译系统 2024 春

实验三

学院:	未来技术学院
姓名:	郭庆吉
学号:	2021113250
指导教师:	朱庆福

一、实验目的

1. 巩固对中间代码生成的基本功能和原理的认识。
2. 能够基于语法指导翻译的知识进行中间代码生成。
3. 掌握类高级语言中基本语句所对应的语义动作。

二、实验环境

Ubuntu 20.04、GCC 11.4、Flex 2.6.4、Bison 3.8.2

三、实验过程

实验内容

1. 中间代码

中间代码是介于源代码和目标机器代码之间的一种表示形式，具有较高的抽象性，不依赖于具体的机器指令集。这样，编译器前端（负责源代码解析和分析）和后端（负责生成目标代码和优化）可以独立开发和优化。不同的后端可以针对不同的目标机器生成合适的机器代码，只需维护一个前端。中间代码比源代码更接近机器级别，但比机器代码更容易进行高层次的优化。由于中间代码与具体的硬件架构无关，因此可以轻松移植到不同的硬件平台。中间代码的设计通常比机器代码更简单，更适合进行自动化处理和生成。

2. 翻译模式

我们将程序中不同的语句进行分类，包括基本表达式，基本语句，声明语句，条件语句，调用语句，数据和结构体的赋值语句等语句。我们为每种语句，每个主要的语法单元设计相应的翻译函数。

3. 数据结构

根据之前实验的词法分析和语法分析我们已经构造除了语法树和符号表，在这个基础上，我们完成了 `intercodegenerate.c` 的设计，即中间代码生成。整体的流程就是将源程序生成的中间代码内容存到 `InterCodes` 这个链表中，所以我们需要做的就是对一些语法单元编写 `translate` 函数，通过语法树根节点 `Program` 的翻译调用到子树，进行递归翻译实现所有中间代码的生成。

本实验中我们需要在实验 2 的基础上额外实现两个数据结构：分别是操作数结构体以及中间代码结构体。前者的作用是保存每个操作数的类型和数值，用来完成中间代码的转换；后者的作用是保存生成的中间代码方便输出。

```
// 操作数（变量，临时变量，标记，函数）
struct Operand_d {
    enum {
        VARIABLE_OP, TEMP_VAR_OP, CONSTANT_OP, LABEL_OP,
        FUNCTION_OP, GET_ADDR_OP, GET_VAL_OP
    } kind;
    union {
        int no; // 临时变量的编号，标记的编号
        int value; // 常量的值
        char name[32]; // 函数以及变量的名字
        Operand opr; // 取地址和解引用指向的操作数
    };
    Type type; // 存放数组/结构体类型变量的类型
    Operand next; // 给argList链接操作数
};

// 单条指令
struct InterCode_d {
    enum {
        LABEL_IR, FUNC_IR, ASSTGN_IR, PLUS_IR, SUB_IR, MUL_IR,
        DIV_IR, TO_MEM_IR, GOTO_IR,
        IF_GOTO_IR, RETURN_IR, DEC_IR, ARG_IR, CALL_IR, PARAM_IR,
        READ_IR, WRITE_IR, NULL_IR
    } kind;
    // 操作数指针数组
    Operand ops[3];
    // 额外信息
    union {
        char relop[32];
        int size;
    };
    // 前一条和后一条指令
    InterCode pre;
    InterCode next;
};
```

4. 翻译过程

全部的语法单元以及基本表达式等内容均对应了一个翻译函数。我们在定义操作的时候先定义一个操作数，然后赋予字段值。然后我们调用中间代码生成函数来生成中间代码，并将其进行存储，利用上文中提到的结构体。然后在最后进行统一输出即可。

下图为条件表达式翻译函数的例子。`translateCond` 函数用于将条件表达式翻译为中间代码。根据条件表达式的不同结构，生成不同的中间代码。`translateProgram` 函数初始化符号表和中间代码，然后开始翻译程序的外部定义列表。

```
// 条件表达式的翻译模式
InterCode translateCond(Node* root, Operand labelTrue, Operand labelFalse) {
    if (root->childNum >= 2 && strcmp(root->children[1]->name, "RELOP") == 0) {
        Operand tmp1 = newTemp();
        Operand tmp2 = newTemp();
        InterCode code1 = translateExp(root->children[0], tmp1);
        InterCode code2 = translateExp(root->children[2], tmp2);
        InterCode code3 = (InterCode)malloc(sizeof(InterCode_));
        code3->kind = IF_GOTO_IR;
        code3->ops[0] = tmp1;
        code3->ops[1] = tmp2;
        code3->ops[2] = labelTrue;
        strcpy(code3->relop, root->children[1]->strVal);
        InterCode code4 = (InterCode)malloc(sizeof(InterCode_));
        code4->kind = GOTO_IR;
        code4->ops[0] = labelFalse;
        insertInterCode(code2, code1);
        insertInterCode(code3, code1);
        insertInterCode(code4, code1);
        return code1;
    }
    else if (strcmp(root->children[0]->name, "NOT") == 0) {
        return translateCond(root->children[1], labelFalse, labelTrue);
    }
    else if (root->childNum >= 2 && (
        strcmp(root->children[1]->name, "AND") == 0 ||
        strcmp(root->children[1]->name, "OR") == 0)) {
        Operand label1 = newLabel();
        InterCode code1;
        switch (root->children[1]->name[0]) {
            case 'A': code1 = translateCond(root->children[0], label1, labelFalse); break;
            case 'O': code1 = translateCond(root->children[0], labelTrue, label1); break;
        }
        InterCode code2 = (InterCode)malloc(sizeof(InterCode_));
        code2->kind = LABEL_IR;
        code2->ops[0] = label1;
        InterCode code3 = translateCond(root->children[2], labelTrue, labelFalse);
        insertInterCode(code2, code1);
        insertInterCode(code3, code1);
        return code1;
    }
    else {
        Operand tmp1 = newTemp();
        InterCode code1 = translateExp(root, tmp1);
        InterCode code2 = (InterCode)malloc(sizeof(InterCode_));
        code2->kind = IF_GOTO_IR;
        code2->ops[0] = tmp1;
        code2->ops[1] = getValue(0);
        code2->ops[2] = labelTrue;
        strcpy(code2->relop, "!=");
        InterCode code3 = (InterCode)malloc(sizeof(InterCode_));
        code3->kind = GOTO_IR;
        code3->ops[0] = labelFalse;
        insertInterCode(code2, code1);
        insertInterCode(code3, code1);
        return code1;
    }
}
```

实验操作流程

在本实验中，我仍然编写了 `MakeFile` 文件进行实验的运行。

首先采用 `make parser` 命令生成中间代码生成器。

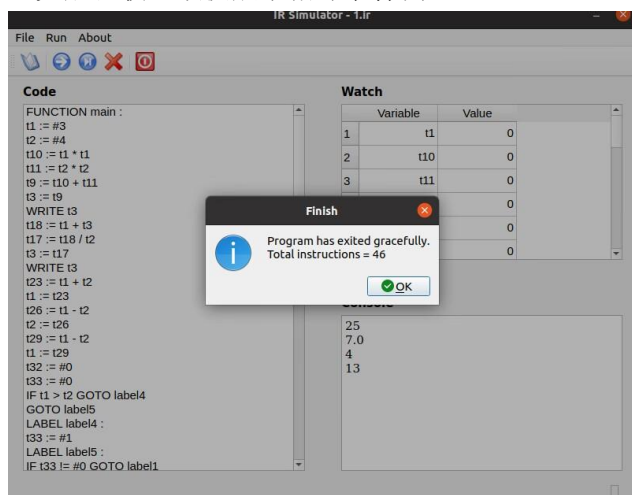
然后使用命令 `./parser test1.cmm test1.ir` 生成对于 `test1.cmm` 代码的中间代码翻译，并存储在 `test1.ir` 文件中，最终使用 `python irsim.py` 命令运行小程序虚拟机，并利用其检验中间代码生成的正确性。命令运行的结果，生成的中间代码在虚拟机小程序中的运行情况均在下一部分中进行体现。

下图展示 Makefile:

```
4 # 编译器设置和编译选项
5 CC = gcc
6 FLEX = flex
7 BISON = bison
8 CFLAGS = -std=c99
9
10 # 编译目标: src目录下的所有.c文件
11 CFILES = $(shell find ./ -name "*.c")
12 OBJS = $(CFILES:.c=.o)
13 LFILE = $(shell find ./ -name "*.l")
14 YFILE = $(shell find ./ -name "*.y")
15 LFC = $(shell find ./ -name "*.l" | sed s[/\./]*/lex.yy.c/)
16 YFC = $(shell find ./ -name "*.y" | sed s[/\./]*/syntax.tab.c/)
17 LFO = $(LFC:.c=.o)
18 YFO = $(YFC:.c=.o)
19
20 parser: syntax $(filter-out $(LFO),$(OBJS))
21      $(CC) -o parser $(filter-out $(LFO),$(OBJS)) -lfl -ly
22
23 syntax: lexical syntax-c
24      $(CC) -c $(YFC) -o $(YFO)
25
26 lexical: $(LFILE)
27      $(FLEX) -o $(LFC) $(LFILE)
28
29 syntax-c: $(YFILE)
30      $(BISON) -o $(YFC) -d -v $(YFILE)
31
32 -include $(patsubst %,%.d, $(OBJS))
33
34 parser_:
35      flex -o ./lex.yy.c ./lexical.l
36      bison -o ./syntax.tab.c -d -v ./syntax.y
37      gcc -g -c ./syntax.tab.c -o ./syntax.tab.o
38      gcc -std=c99 -g -c -o intercode.o intercode.c
39      gcc -std=c99 -g -c -o semantic.o semantic.c
40      gcc -std=c99 -g -c -o Tree.o Tree.c
41      gcc -std=c99 -g -c -o main.o main.c
42      gcc -g -o parser ./intercode.o ./semantic.o ./Tree.o ./syntax.tab.o ./main.o -lfl -ly
43
44 # 定义的一些伪目标
45 .PHONY: clean test
46 test: parser_
47      ./parser ../Test/self_test.cmm out.ir
48 clean:
49      rm -f parser lex.yy.c syntax.tab.c syntax.tab.h syntax.output
50      rm -f $(OBJS) $(OBJS:.o=.d)
51      rm -f $(LFC) $(YFC) $(YFC:.c=.h)
52      rm -f *
```

实验结果

已完成验收，故仅展示指导书样例 1:



四、实验总结

本次实验让我更加深刻的理解了中间代码的作用和生成过程，并且在小程序执行中间代码的运行过程中深化了中间代码在编译系统中桥梁的作用。